LAB 7

Fuzzing for Input Validation Bugs

Coupon Validator Input-Processing Application

Student Name	Mohammed Shahid Javed Mokashi
SRN	PES1UG24AM378
Course	Software Engineering Laboratory
Assigned repository	PES1UG24AM378/L6_L7_31_coupon_validator
Deliverables folder	Lab7_Deliverables

1. Objective

The lab explores fuzz testing as a security and quality-assurance technique by identifying and fixing bugs in a Python application using the Hypothesis property-based fuzzing library. The learning outcomes include understanding fuzzing, applying property-based tests, analysing edge-case failures, and improving robustness through defensive programming.

2. Application and functions

The assigned repository contains three functions. `split_code()` parses a coupon in PREFIX-NUMBER form. `most_expensive()` finds the highest price in comma-separated input. `expiry_days()` parses a YYYY-MM-DD date and calculates its day offset from 2024.

3. Bugs identified in the starter implementation

The original implementation assumes all input is well-formed. It can leak AttributeError for non-string values, IndexError for missing coupon separators, ValueError for empty or non-numeric prices, and it does not validate real calendar dates. Its date calculation also approximates every month as 30 days and every year as 365 days, so it is inaccurate around month lengths and leap years. This is visible in the evidence below: for 2025-12-31 the original returns 726 days, while the correct offset from 2024-01-01 is 730.

4. Defensive fixes

The updated application checks input types and formats before processing. Coupon codes must contain one non-empty alphanumeric prefix and a non-negative integer suffix. Price lists must be non-empty and contain only finite, non-negative numbers. Dates must be real calendar dates in strict YYYY-MM-DD format and are calculated using Python date arithmetic, including leap years. Invalid types raise TypeError; invalid values raise ValueError.

5. Hypothesis test design

The test suite (`test_processor.py`, 12 tests) uses Hypothesis strategies for arbitrary Unicode strings, None, generated coupon prefixes and integers, lists of floating-point prices, and generated calendar dates. It asserts that inputs either produce valid results or are rejected with the documented TypeError/ValueError exceptions. Focused tests cover empty fields, extra separators, malformed prices, non-finite values, negative prices, invalid dates, and None.

6. Execution commands

```
py -m pip install -r requirements.txt
py original_processor.py
py -m pytest -v test_processor.py
py processor.py
py -m pytest -v test_processor.py
```

7. Evidence

All screenshots below are captured from the author's local PowerShell session in `D:\Lab7_Deliverables` (Python 3.12.5, pytest 8.4.1, Hypothesis 6.170.15). Figures 1 and 2 show the original implementation; Figures 3 and 4 show the fixed implementation.

Figure 1 — Original application output

Original `original_processor.py` with normal sample input. Note expiry_days returns 726 for 2025-12-31 because of the 30-day-month / 365-day-year approximation.

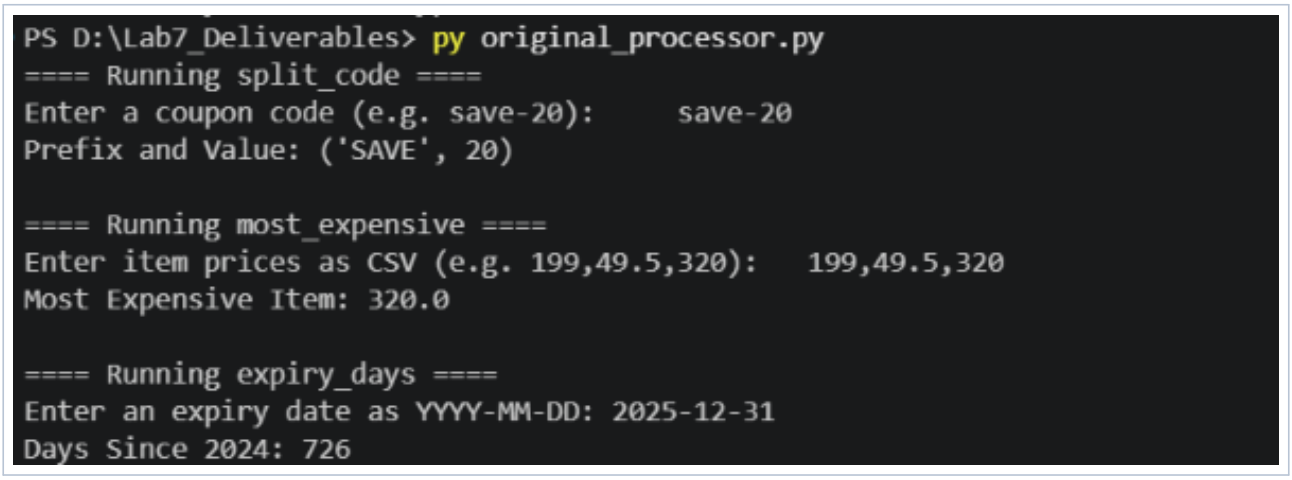


Figure 2 — Hypothesis tests failing on the original code

`py -m pytest -v test_processor.py` against the original implementation: 10 of the 12 tests fail, exposing the unhandled edge cases (arbitrary text, missing/extra parts, empty or non-numeric prices, invalid dates, None). Only the two valid-input generators for split_code and most_expensive pass.

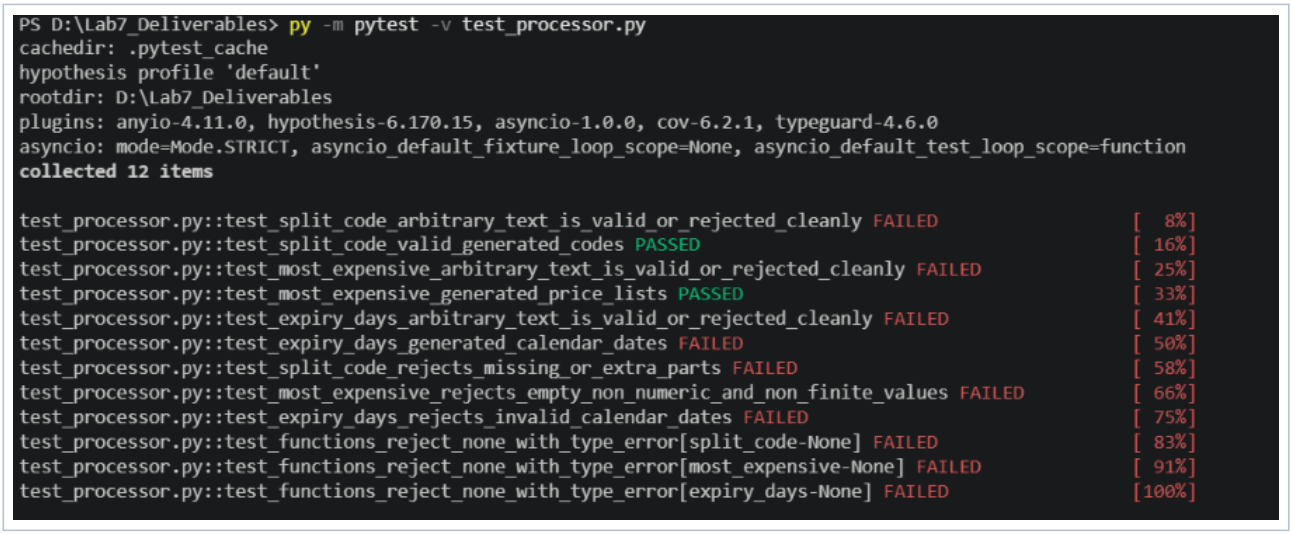


Figure 3 — Fixed application output

The fixed `processor.py` handles the same sample inputs; expiry_days now returns the correct 730 days since 2024-01-01.

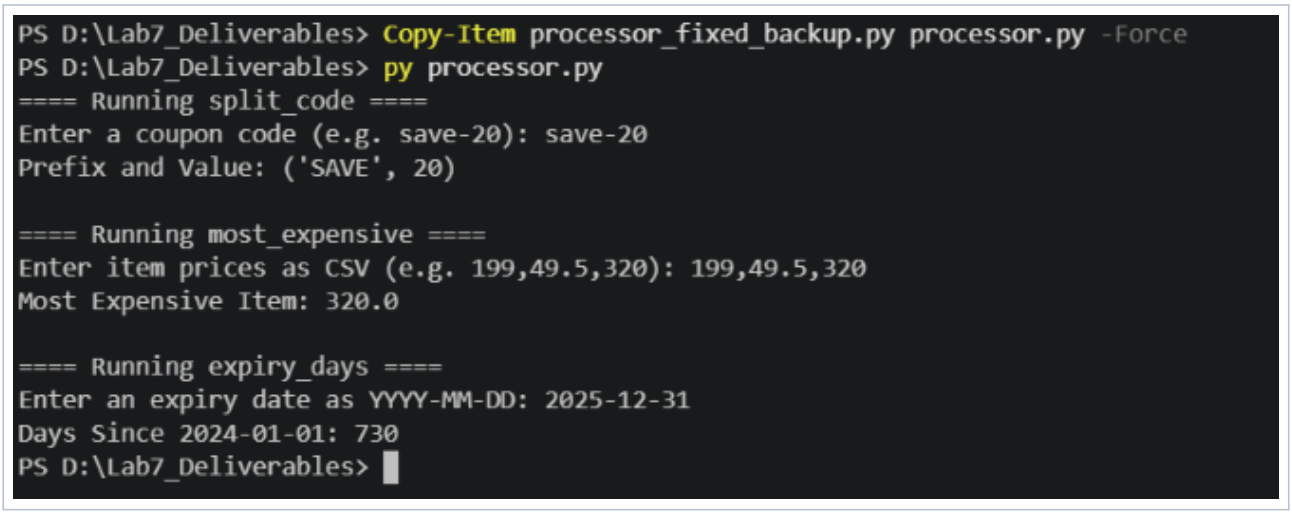
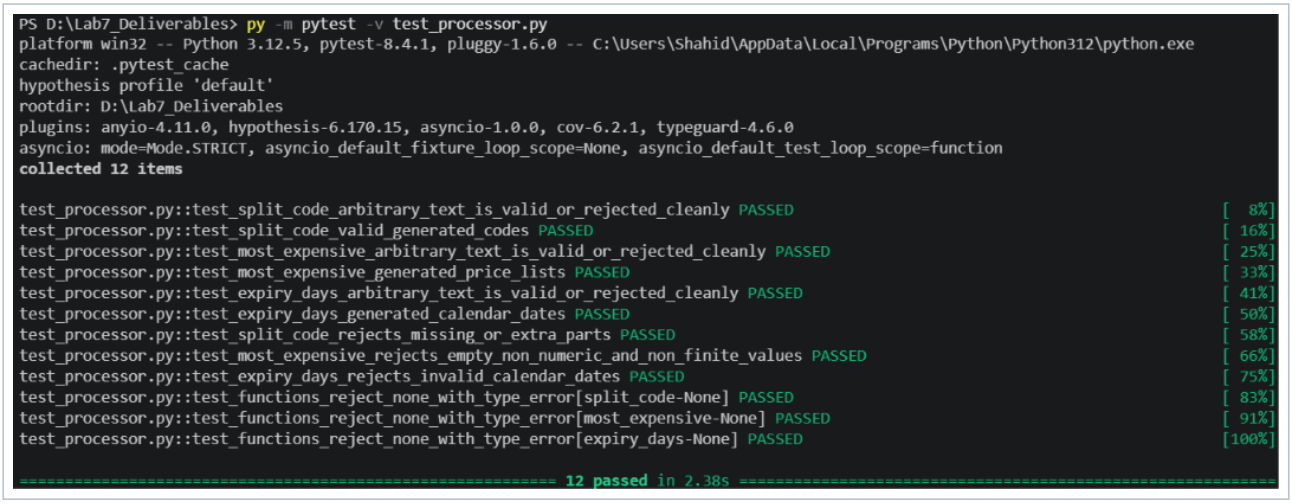


Figure 4 — Hypothesis tests passing on the fixed code

`py -m pytest -v test_processor.py` against the fixed implementation: all 12 tests pass in 2.38 s.



8. Conclusion

Fuzzing with Hypothesis exposed multiple input-validation weaknesses in the starter code: 10 of 12 property and edge-case tests failed before the fix. After adding explicit input contracts and replacing the approximate date arithmetic with Python's calendar-aware date handling, all 12 tests pass. Malformed input is now rejected through the documented TypeError/ValueError contract rather than leaking incidental implementation exceptions, and the date calculation is correct across month lengths and leap years.

Submission checklist

1. PDF report — included
2. Fixed application script (processor.py) — included
3. Updated Hypothesis test script (test_processor.py) — included
4. Hypothesis failing-before and passing-after terminal screenshots — included (Figures 2 and 4)